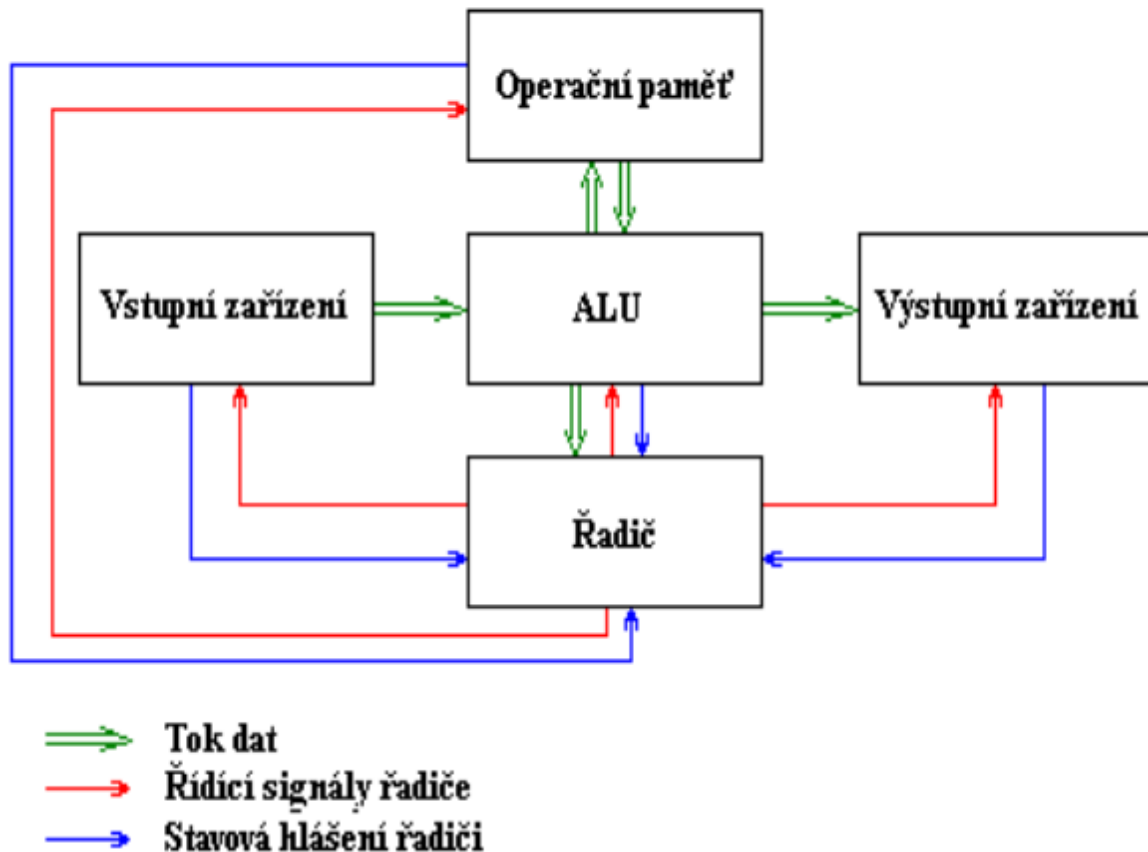


Obsah

1	Von Neumannovo schéma.....	2
2	Princip činnosti.....	3
3	Odlišnosti dnešních počítačů.....	4
4	BIOS (Basic Input / Output System) ROM (Read Only memory), FLASH.....	5
5	CMOS (Complementary Metal Oxide Silicon).....	6
6	Blokové schéma procesoru.....	7
7	RISC.....	8
8	CISC.....	9
8.1	Příklady CISC procesorů:.....	9
9	Typy instrukcí.....	10
	BX, známý jako bazový registr, má několik důležitých funkcí v architektuře x86:.....	12
10	Příklad: Sečtení dvou čísel.....	15
11	Fáze zpracování instrukce.....	18
12	Dnes.....	19

1 Von Neumannovo schéma

Von Neumannovo schéma



Bylo navrženo 1945 Johnem von Neumannem (maďarský původ).

Podle tohoto schématu se počítač skládá z pěti hlavních modulů:

Operační paměť: slouží k uchování zpracovávaného programu, zpracovávaných dat a výsledků výpočtu.

ALU – Arithmetic-Logic Unit (aritmetickologická jednotka): jednotka provádějící veškeré aritmetické výpočty a logické operace. Obsahuje sčítačky, násobičky (pro aritmetické výpočty) a komparátory (pro porovnávání).

Řadič: řídicí jednotka, která řídí činnost všech částí počítače. Toto řízení je prováděno pomocí řídicích signálů, které jsou zasílány jednotlivým modulům. Reakce na řídicí signály, stavy jednotlivých modulů jsou naopak zasílány zpět řadiči pomocí **stavových hlášení**.

Vstupní zařízení: zařízení určená pro vstup programu a dat.

Výstupní zařízení: zařízení určená pro výstup výsledků, které program zpracoval.

2 Princip činnosti

Do operační paměti se pomocí vstupních zařízení umístí program (posloupnost operací – instrukcí), který se bude vykonávat.

Stejným způsobem se do operační paměti umístí data, která bude program zpracovávat

Proběhne vlastní výpočet, jehož jednotlivé kroky provádí ALU.

Tato jednotka je v průběhu výpočtu spolu s ostatními moduly řízena řadičem počítače.

Mezivýsledky výpočtu jsou ukládány do operační paměti.

Po skončení výpočtu (i dílčího) mohou být výsledky poslány na výstupní zařízení.

3 Odlišnosti dnešních počítačů

Podle von Neumannova schématu počítač pracuje vždy nad jedním programem. Toto vede k velmi špatnému využití strojového času.

Dnešní počítače zpracovávají paralelně více programů najednou – tzv. multitasking.

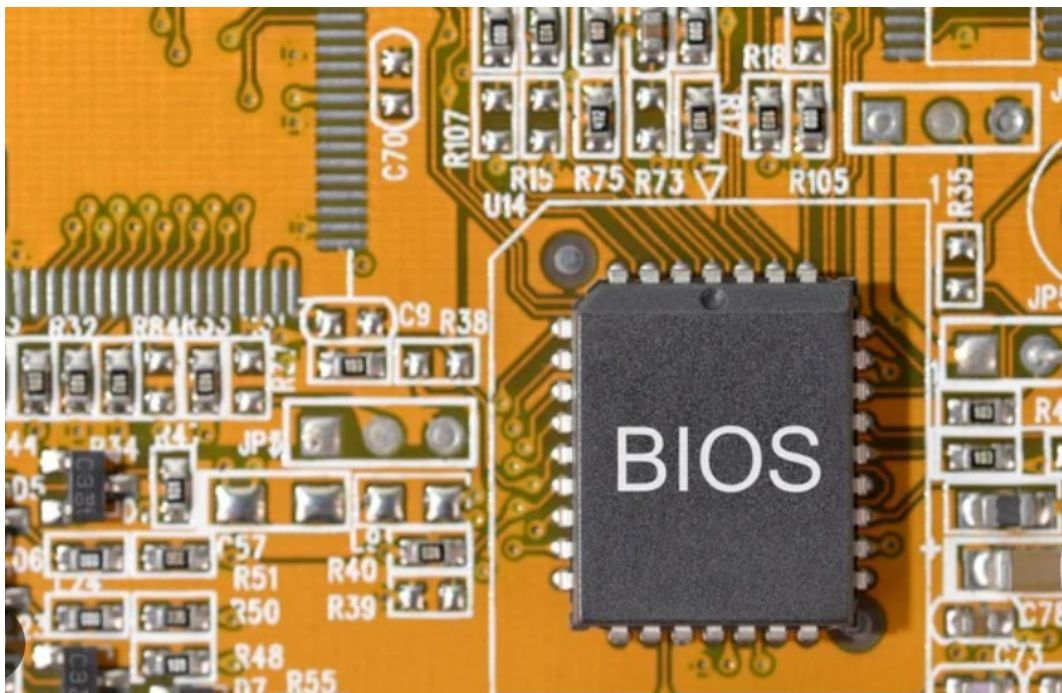
Počítač může disponovat i více než jedním procesorem.

Počítač podle von Neumannova schématu pracoval pouze v tzv. diskrétním režimu (po spuštění programu není možnost komunikace).

Existují vstupní / výstupní zařízení (I/O devices), která umožňují jak vstup, tak výstup dat (programu). Program se do paměti nemusí zavést celý, ale je možné zavést pouze jeho část a ostatní části zavádět až v případě potřeby.

4 BIOS (Basic Input / Output System) ROM (Read Only memory), FLASH

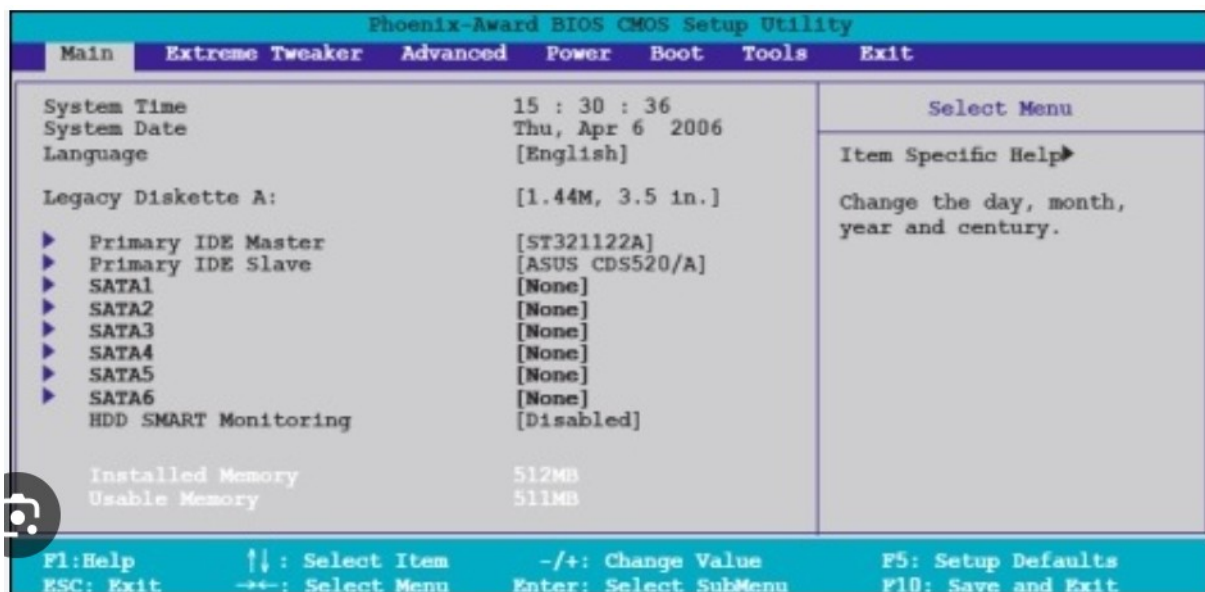
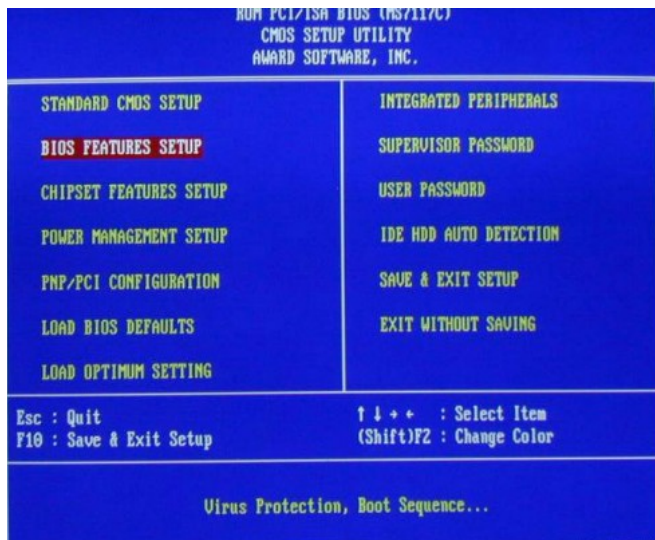
- Základní programové vybavení každého PC
- BIOS, POST, zavaděč a spouštěč OS, SETUP
- Program přizpůsobený danému HW konkrétního počítače
- Poskytuje jednotné SW rozhraní pro konkrétní počítač a tím zabezpečuje snadnou přenositelnost ostatního SW
- Základní BIOS – umístěn v ROM zákl. desky nebo ve flash
- Služby BIOSu, jsou používány OS a programátory aplikací
- Po vyvolání přerušení se provede příslušná obslužná rutina
- AMI, AWARD, PHOENIX



5 CMOS (Complementary Metal Oxide Silicon)

Paměť typu RAM

- Data o konfiguraci počítače
- Pro editaci slouží SETUP, umožňující konfiguraci počítače
- Zálohována baterií
- Integrace obvodu hodin reálného času do CMOS
- Paměť typu RAM - informace, které nemohou být na disku ani v RAM

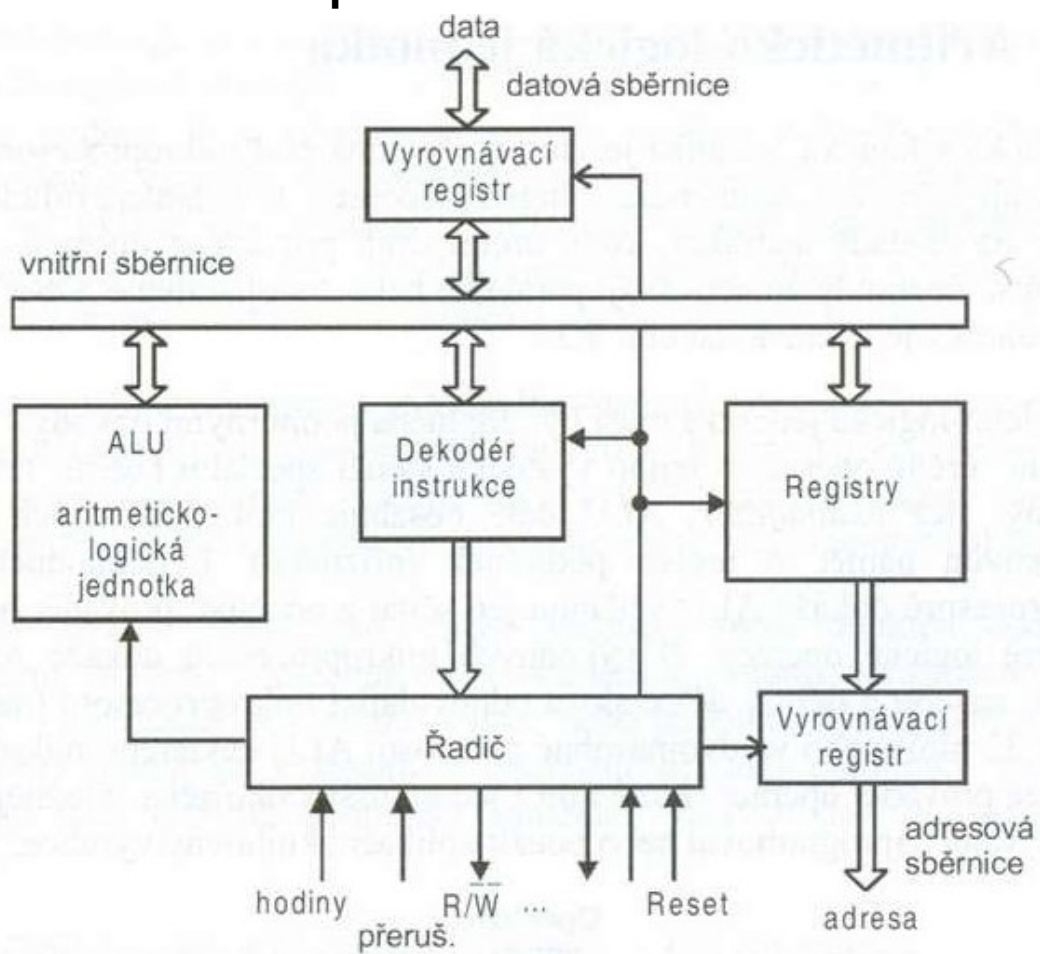


6 Procesor

Procesor je hlavní elektronická součástka v počítači, která vykonává instrukce počítačového programu tak, že provádí základní aritmetické, logické, kontrolní a vstupní/výstupní operace specifikované instrukcemi.



7 Blokové schéma procesoru



Řadič

Určuje zda je procesor

- RISC (Reduced Instruction Set Computing)
- CISC (Complex Instruction Set Computing)

8 RISC

RISC (Reduced Instruction Set Computer) je typ architektury procesoru, který používá jednoduchou a optimalizovanou sadu instrukcí. Hlavní myšlenkou RISC je zjednodušení instrukční sady, což umožňuje rychlejší a efektivnější zpracování instrukcí. Zde jsou některé klíčové vlastnosti RISC:

1. **Jednoduché instrukce:** Instrukce jsou navrženy tak, aby byly jednoduché a rychlé na vykonání. Každá instrukce obvykle trvá jeden cyklus procesoru.
2. **Pipelining:** RISC procesory často využívají pipelining, což umožňuje paralelní zpracování instrukcí a zvyšuje výkon.
3. **Velký počet registrů:** RISC procesory mají obvykle více registrů, což minimalizuje potřebu přístupu k paměti a zvyšuje rychlost zpracování.
4. **Optimalizace kompilátoru:** Kompilátory pro RISC procesory jsou navrženy tak, aby efektivně využívaly jednoduchou instrukční sadu a optimalizovaly výkon.

Výhody RISC:

- **Vyšší výkon:** Díky jednoduchým instrukcím a pipeliningu mohou RISC procesory dosáhnout vyššího výkonu.
- **Efektivita:** Jednoduchá instrukční sada umožňuje efektivnější využití zdrojů procesoru.
 - **ARM:** Velmi populární architektura používaná v mobilních zařízeních, jako jsou smartphony a tablety.
 - **MIPS:** Používá se v různých vestavěných systémech a síťových zařízeních.
 - **SPARC:** Vyvinutý společností Sun Microsystems, používá se v serverech a pracovních stanicích.
 - **Power Architecture:** Používá se v serverech, superpočítačích a některých herních konzolách.
 - **Alpha:** Vyvinutý společností Digital Equipment Corporation (DEC), používán v high-performance computing.
 - **PA-RISC:** Vyvinutý společností Hewlett-Packard, používán v jejich pracovních stanicích a serverech.
 - **AVR:** Používá se v mikrokontrolérech, často v embedded systémech.
 - **PIC:** Další typ mikrokontroléru používaný v embedded systémech.
 - **ARC:** Používá se v různých vestavěných aplikacích

- V řadiči je zakódovaný strojový kód procesoru potažmo počítače
- Různé řešení – Sekvenční automat – Mikroprogramový řadič – ...
- pipelining

Pipelining je technika používaná v procesorech ke zvýšení jejich výkonnosti. Umožňuje rozdělit instrukce na několik fází, které mohou být zpracovávány paralelně. Představte si to jako výrobní linku, kde každá fáze zpracovává jinou část instrukce.

Zde je základní přehled, jak pipelining funguje:

1. **Fetch (načtení):** Načtení instrukce z paměti.
2. **Decode (dekódování):** Překlad instrukce do formátu, kterému procesor rozumí.
3. **Execute (provedení):** Vykonání instrukce.
4. **Memory (paměť):** Přístup k paměti, pokud je to potřeba.
5. **Write-back (zápis výsledků):** Zápis výsledků zpět do registrů.

9 CISC

CISC (Complex Instruction Set Computer) je typ architektury procesoru, který používá rozsáhlou a komplexní sadu instrukcí. Hlavní myšlenkou CISC je, že procesor může vykonávat složité operace pomocí jediné instrukce, což může zjednodušit programování a snížit počet instrukcí potřebných k vykonání určitého úkolu.

Zde jsou některé klíčové vlastnosti CISC:

1. **Komplexní instrukce:** Instrukce mohou vykonávat složité operace, jako je načítání dat z paměti, provádění aritmetických operací a ukládání výsledků zpět do paměti, vše v jedné instrukci.
2. **Variabilní délka instrukcí:** Instrukce mohou mít různou délku, což umožňuje větší flexibilitu, ale také zvyšuje složitost dekódování instrukcí.
3. **Menší počet registrů:** CISC procesory obvykle mají méně registrů než RISC procesory, protože mnoho operací může být prováděno přímo v paměti.
4. **Mikrokódování:** Komplexní instrukce jsou často rozděleny na jednodušší mikroinstrukce, které jsou vykonávány interně procesorem.

9.1 Příklady CISC procesorů:

- **Intel x86:** Jedna z nejznámějších a nejrozšířenějších CISC architektur, používaná v osobních počítačích.
- **AMD64:** Rozšíření x86 architektury, které přidává podporu pro 64bitové instrukce.

10 Typy instrukcí

- Aritmetické - sčítání, odčítání, násobení a dělení.
- Logické - AND, OR, XOR a NOT, které manipulují s jednotlivými bity dat
- Posuvy a rotace - mění pozici bitů v registru, což může být užitečné pro rychlé násobení nebo dělení čísla dvěma.
- Přesuny - kopírují data z jednoho místa do druhého, například z registru do paměti nebo mezi registry.
- Instrukce skoků
 - nepodmíněný - JMP adresa - Přesune řízení programu na specifikovanou adresu bez ohledu na jakékoli podmínky.
 - podmíněný - JNZ adresa Přesune řízení programu na specifikovanou adresu pouze pokud je splněna určitá podmínka, například pokud je hodnota v registru nenulová.
- Řídící instrukce - mění způsob, jakým procesor vykonává instrukce, například instrukce pro volání podprogramů nebo návrat z podprogramů.
- I/O instrukce - umožňují komunikaci mezi procesorem a periferními zařízeními, jako jsou klávesnice, displeje nebo síťové karty.

Registry procesoru x86

aritmetické

- AX – akumulátor (řada instrukcí ho má jako implicitní operand)
 - BX, známý jako bázeový registr, má několik důležitých funkcí v architektuře x86:
 - CX – čítač (tj. určený pro počítání cyklů)
 - DX – rozšíření akumulátoru (někdy nazýván data register)
 - SI – source index – index pro zdroj (tj. pro čtení)
 - DI – destination index – index pro cíl (tj. pro zápis)
 - BP – base pointer – určen jako ukazatel na záznam aktivní procedury na zásobníku (tím, že se implicitně spojoval s SS) – SP – stack pointer – ukazatel vrcholu zásobníku
- segmentové
 - CS – segment kódu
 - DS – datový segment
 - ES – extra segment
 - SS – zásobníkový (stack) segment
 - registr IP (instrukční pointer neboli čítač instrukcí), vždy odkazovaný implicitně
 - registr FLAGS (příznaky)

Poznámky

AX

Akumulátor, často označovaný jako registr AX v architektuře x86, je speciální registr používaný pro různé aritmetické a logické operace. Slouží jako hlavní registr pro ukládání mezivýsledků při provádění operací. Zde jsou některé konkrétní funkce akumulátoru AX:

1. **Aritmetické operace:** AX je často používán jako operand pro instrukce jako ADD, SUB, MUL, a DIV.
2. **Logické operace:** AX může být použit pro operace jako AND, OR, XOR, a NOT.
3. **I/O operace:** AX je často používán při instrukcích pro vstup a výstup dat, například při komunikaci s periferními zařízeními.
4. **Přesuny dat:** AX může být použit pro přesun dat mezi pamětí a jinými registry.
5. **Řídící instrukce:** AX může být použit při volání podprogramů nebo návratu z podprogramů.

BX, známý jako bazový registr, má několik důležitých funkcí v architektuře x86:

1. **Indexování paměti:** BX se často používá jako bazový registr pro adresování paměti, což umožňuje přístup k datům v paměti pomocí relativních adres.
2. **Přesuny dat:** BX může být použit pro přesun dat mezi pamětí a jinými registry.
3. **Podpora pro složené adresování:** BX může být kombinován s dalšími registry (např. SI, DI) a offsety pro složené adresování, což je užitečné při práci s datovými strukturami jako jsou pole a tabulky.
4. **Aritmetické operace:** BX může být použit jako operand pro aritmetické operace, podobně jako AX.

FLAG

Registr FLAGS (nebo také stavový registr) je speciální registr v procesoru, který uchovává různé stavové bity, které odrážejí výsledky aritmetických a logických operací. Tyto bity jsou používány pro řízení toku programu a rozhodování. Zde jsou některé z hlavních funkcí a příkladů stavových bitů v registru FLAGS:

1. **Zero Flag (ZF):** Nastaven, pokud je výsledek operace nula.
2. **Sign Flag (SF):** Nastaven, pokud je výsledek operace záporný.
3. **Overflow Flag (OF):** Nastaven, pokud došlo k přetečení při aritmetické operaci se znaménkem.
4. **Carry Flag (CF):** Nastaven, pokud došlo k přenosu nebo zápůjčce při aritmetické operaci.
5. **Parity Flag (PF):** Nastaven, pokud je počet jedničkových bitů ve výsledku sudý.
6. **Auxiliary Carry Flag (AF):** Používán při operacích s BCD (Binary-Coded Decimal) aritmetikou.

Registr IP

Registr IP (Instruction Pointer) je speciální registr v procesoru, který uchovává adresu následující instrukce, která má být vykonána. Je klíčový pro řízení toku programu. Zde jsou některé hlavní funkce registru IP:

1. **Řízení toku programu:** IP určuje, kterou instrukci procesor vykoná jako další, což umožňuje sekvenční vykonávání programu.
2. **Skoky a volání podprogramů:** Při skocích (JMP) nebo volání podprogramů (CALL) se hodnota IP změní na adresu cílové instrukce, což umožňuje přechod na jiné části programu.

3. **Přerušení:** Při přerušení se hodnota IP uloží a procesor přejde na obsluhu přerušení, po jejímž dokončení se IP obnoví a pokračuje v původním programu.

Ukázka instrukcí

- `add r0, r2;` – sčítání - přičte do registru r0 hodnotu uloženou v registru r2
- `addc r1, r3;` – sčítání s přenosem - přičte do registru r1 hodnotu registru r3 a příznak přenosu C
- `mov 1234h, r0;` – přesun - uloží do paměti na adresu 1234h hodnotu z registru r0
- `mov 1236h, r1;` – přesun - uloží do paměti na adresu 1236h hodnotu z registru r1
- `mov [r7], r0;` – přesun - uloží na adresu určenou registrem r7 hodnotu z registru r0
- `cmp r4, r5;` – porovnání - porovná hodnoty registrů r4 a r5
- `jmp r cc_UGT, 100h;` – podmíněný relativní skok - pokud byla hodnota registru r4 vyšší, pokračuje program o 100h adres dále ($PC \leftarrow PC + 100h$)

11 Příklad: Sečtení dvou čísel

Předpokládejme, že chceme sečíst dvě čísla, která jsou uložena v registrech AX a BX, a výsledek uložit do registru AX.

Předpokládejme, že chceme sečíst dvě čísla, která jsou uložena v registrech AX a BX, a výsledek uložit do registru AX.

```
section .data
```

```
    ; Data section (není potřeba pro tento jednoduchý příklad)
```

```
section .text
```

```
    global _start
```

```
_start:
```

```
    ; Předpokládejme, že čísla jsou již v registrech AX a BX
```

```
    mov AX, 5    ; Uložení čísla 5 do registru AX
```

```
    mov BX, 3    ; Uložení čísla 3 do registru BX
```

```
    ; Sečtení čísel
```

```
    add AX, BX   ; AX = AX + BX
```

```
    ; Výsledek je nyní v registru AX (AX = 8)
```

```
    ; Ukončení programu (pro Linux)
```

```
    mov eax, 60  ; syscall číslo pro exit
```

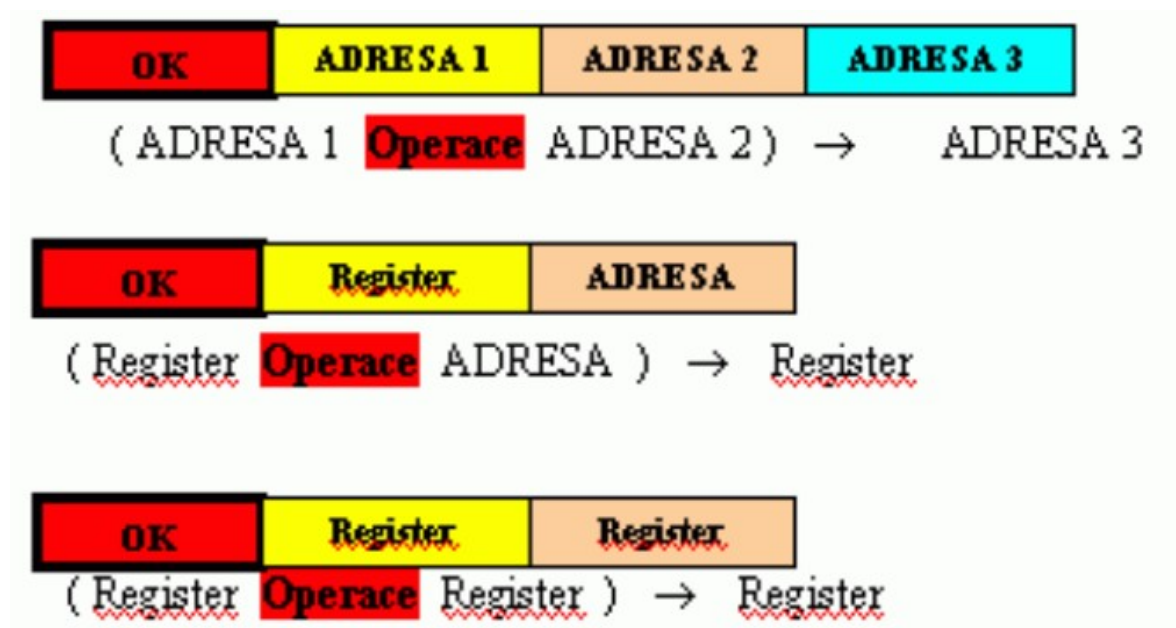
```
    xor edi, edi ; exit code 0
```

```
    syscall
```

Vysvětlení kroků:

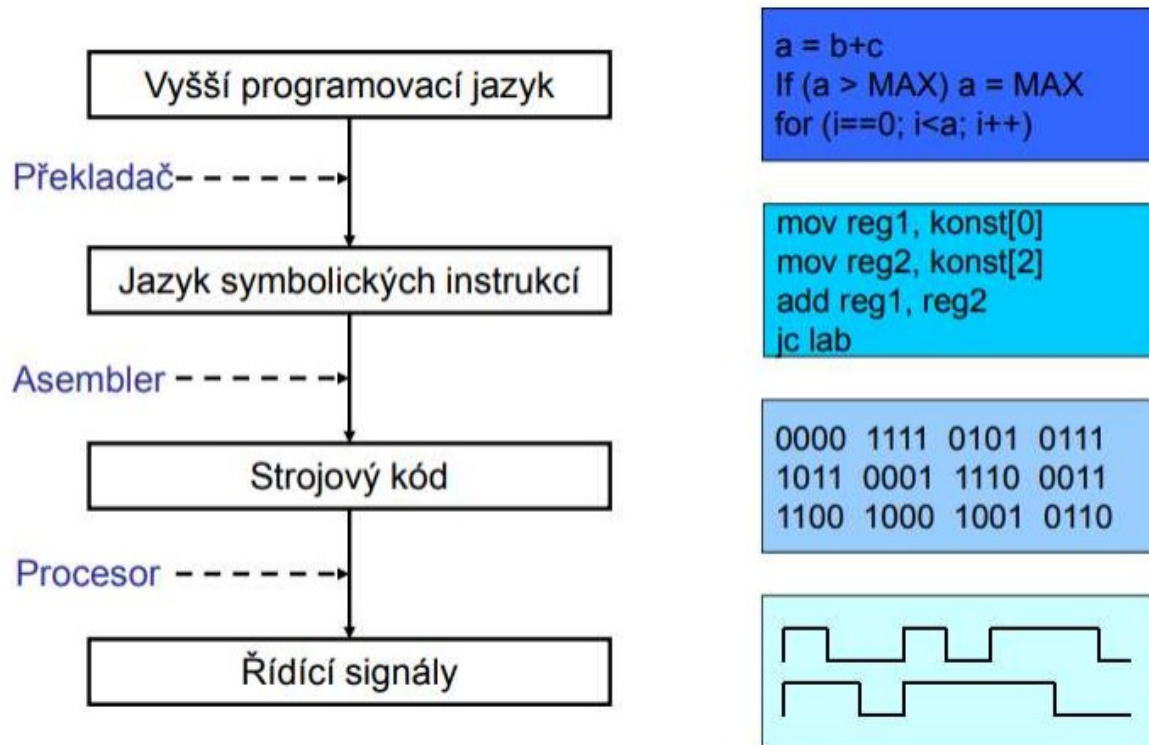
1. **Inicializace registrů:** Uložíme čísla 5 a 3 do registrů AX a BX.
2. **Sečtení čísel:** Použijeme instrukci `add`, která sečte hodnoty v registrech AX a BX a uloží výsledek zpět do registru AX.
3. **Ukončení programu:** Použijeme systémové volání `exit` k ukončení programu.

Ukázka možné struktury instrukcí



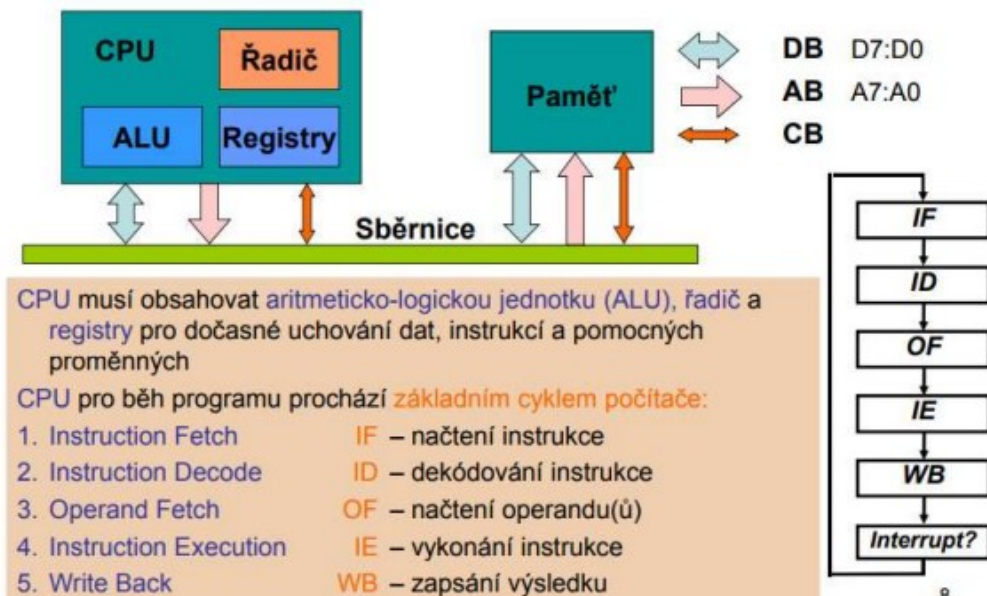
OK – operační kód (binární kód instrukce)

Program



12 Fáze zpracování instrukce

DB – DataBus (datová část sběrnice);
 AB – AdressBus (adresová část sběrnice);
 CB – ControlBus (řídící část sběrnice);



13 Dnes

Mikrokód je u procesorů Intel (Pentium Pro a novějších) možné nahrazovat, čehož využívají výrobci procesorů k opravě chyb. Nový mikrokód se musí po každém zapnutí nahrávat znovu, protože je uložen ve volatilní paměti a mikroprocesor se po vypnutí vrací ke svému původnímu mikrokódu. K aktualizaci mikrokódu může být použit BIOS (v rámci POST testů po zapnutí počítače) nebo je možné ho nahradit i později.